

# ShopEz

## Full Stack MERN E-Commerce Application

### Project Technical Documentation

**Developed by:** Turlapati Lahari

**Department:** Data Science

**Email:** 24eg109a54@anurag.edu.in

**Team ID:** \_\_\_\_\_

**Date:** 18 June 2026

# 1. Introduction

**Project Title:** ShopEz

**Lead Developer:** Turlapati Lahari (Solo Project)

ShopEz is a robust, full-stack digital commerce platform engineered to deliver an streamlined and modern shopping experience. Developing with the MERN architectural stack (MongoDB, Express, React, and Node.js), this application provides a sophisticated interface connecting consumers with product catalogs through a device-agnostic, responsive design framework.

## 2. Project Overview

**Purpose:** The goal of this undertaking is to construct a scalable and secure e-commerce ecosystem. It allows end-users to navigate categorized inventories, manage real-time shopping carts, and finalize transactions safely, while simultaneously providing administrative tools for business management.

**Core Functionalities:**

- **User Core:** Secure registration, JWT-based login sessions, and personalized profiles.
- **Inventory Core:** Dynamic discovery modules (Men, Women, Kids) with integrated search and data filtering.
- **Transaction Core:** Real-time cart state management and Cash on Delivery (COD) processing logic.
- **Admin Hub:** Integrated dashboard for managing product data and tracking order fulfillment status.

## 3. System Architecture

**Frontend (React.js):** Utilizes a modular Component-Based Architecture. Global state is managed via the React Context API to ensure data flow consistency, while React Router DOM handles client-side navigation without page reloads.

**Backend (Node.js & Express.js):** An MVC-structured RESTful API. Express handles the routing layer, while custom middleware manages user authentication and payload validation. The server provides structured JSON responses to the client application.

**Database (MongoDB):** A NoSQL document database. Key data structures include:

- **Users:** Secure storage for credentials, contact details, and user roles.
- **Products:** Rich attributes including name, category, pricing, and stock levels.
- **Orders:** Tracking for history, item references, and current shipping status.

## 4. Setup Instructions

**Prerequisites:** Node.js (v18+), NPM/Yarn, and a running MongoDB instance.

### 1. Repository Initialization:

```
git clone https://github.com/24eg109a54-lahari/VIP_C2_ShopEz_E-Commerce_application_by_-Turlapati-Lahari.git
```

**2. Server Configuration:** Load dependencies in the `/server` folder and create a `.env` file containing the `PORT`, `MONGO_URI`, and `JWT_SECRET`.

```
cd server && npm install
```

**3. Client Configuration:** Navigate to `/client` and initialize the frontend environment.

```
cd client && npm install
```

## 5. Folder Structure

| Directory                           | Functional Description                                     |
|-------------------------------------|------------------------------------------------------------|
| <code>/server/models</code>         | Mongoose schemas defining the structure of the data layer. |
| <code>/server/routes</code>         | Secure API endpoints for client-server communication.      |
| <code>/client/src/components</code> | Modular UI foundations (Navbar, Product Cards, etc).       |
| <code>/client/src/pages</code>      | View-level components (Home, Checkout, Admin).             |

## 6. Running the Application

Initiate the development environment by running two parallel terminal instances:

**Backend:** `cd server && npm run dev`

**Frontend:** `cd client && npm start`

## 7. API Specification

| Method | Endpoint         | Description                                                  |
|--------|------------------|--------------------------------------------------------------|
| POST   | /api/users/login | Authenticates users and allocates JWT security tokens.       |
| GET    | /api/products    | Retrieves the product catalog with optional sorting filters. |

## 8. Authentication & Authorization

Security is enforced through JSON Web Tokens (JWT) for stateless requests. Tokens generated upon login are stored in the browser's LocalStorage and passed in the 'Authorization' header. Custom middleware validates these tokens and checks user roles specifically for admin-only routes.

## 9. User Interface

The interface is designed with a minimalist aesthetic for optimal usability. Navigation is streamlined through category-specific grids and a multi-step checkout funnel. High-resolution media assets and product banners are integrated with responsive visual placeholders.

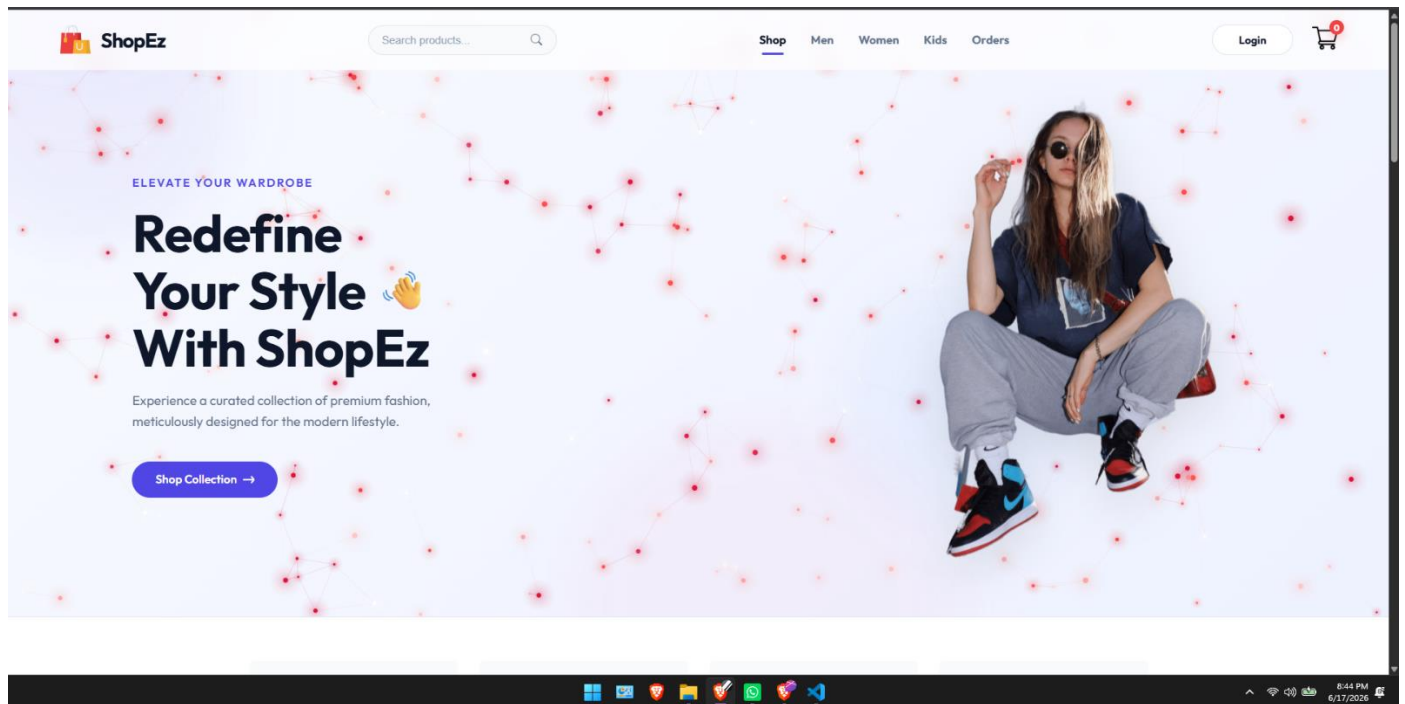
## 10. Testing Methodology

A rigorous testing framework was employed to ensure application stability:

- **Unit Testing:** Validated individual logic functions and form validators.
- **Integration Testing:** Verified that Express routes correctly communicate with MongoDB.

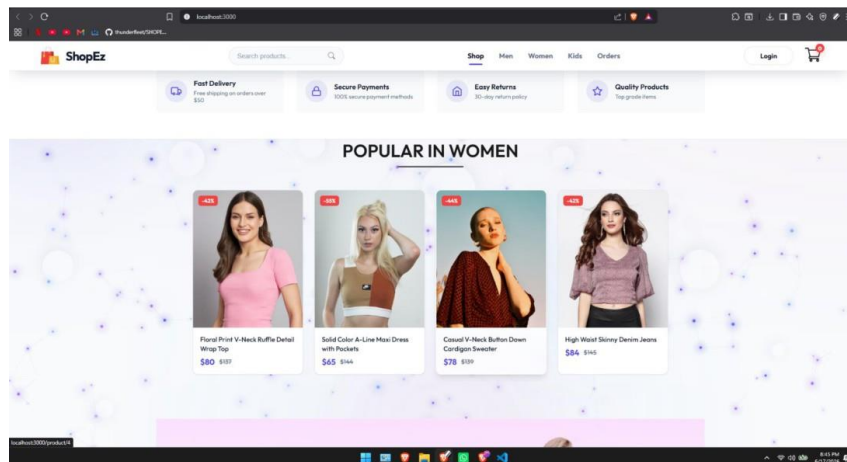
## 11. Screenshots / Demo

Repository      Access:      [https://github.com/24eg109a54-lahari/VIP\\_C2\\_ShopEz\\_E-Commerce\\_application\\_by\\_-Turlapati-Lahari.git](https://github.com/24eg109a54-lahari/VIP_C2_ShopEz_E-Commerce_application_by_-Turlapati-Lahari.git)

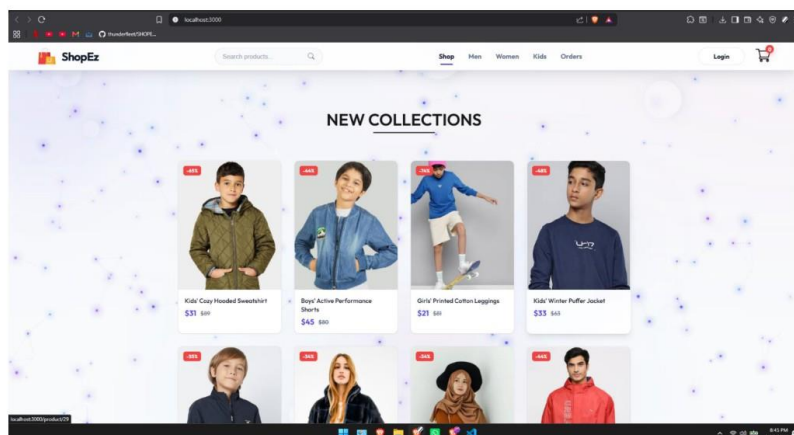


Application Web Interface - Home Landing Page

Product Category – Women's Fashion  
Product Category - Women's Fashion



New Collections Section  
New Collections Section

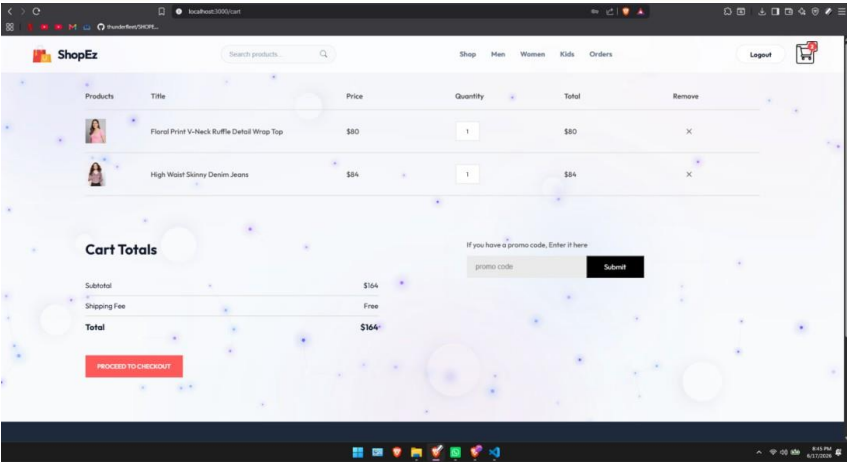


Page 6

Dynamic Product Catalog - Filtered View

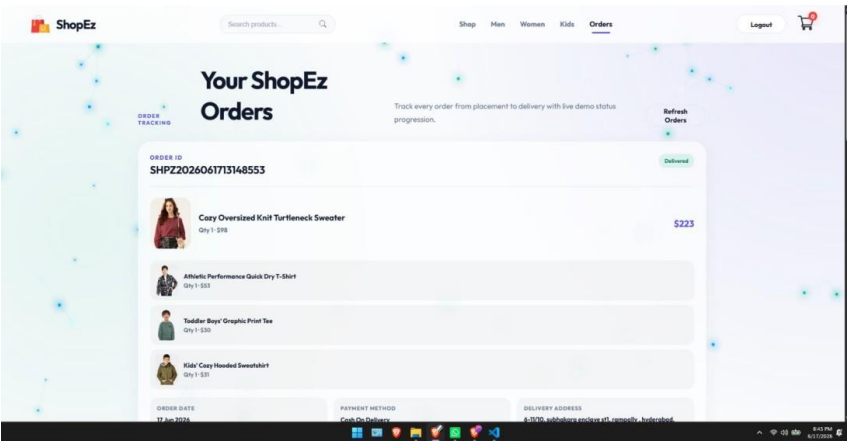
Cart & Checkout Interface

*Cart & Checkout Interface*



User Orders History

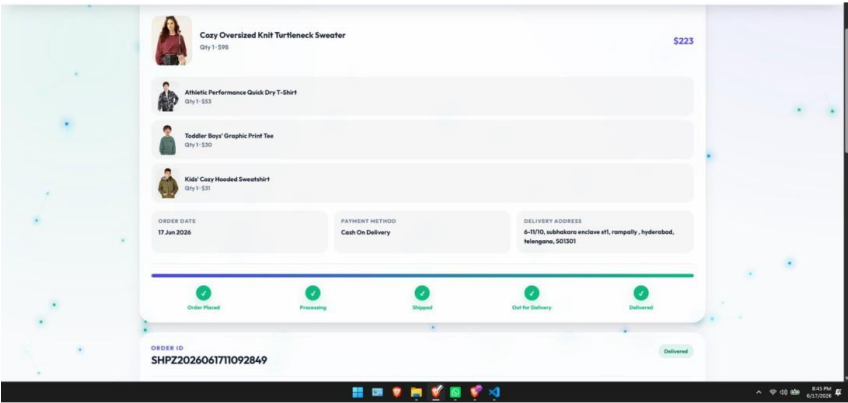
*User Orders History*



Real-time Cart Management & Checkout Pipeline

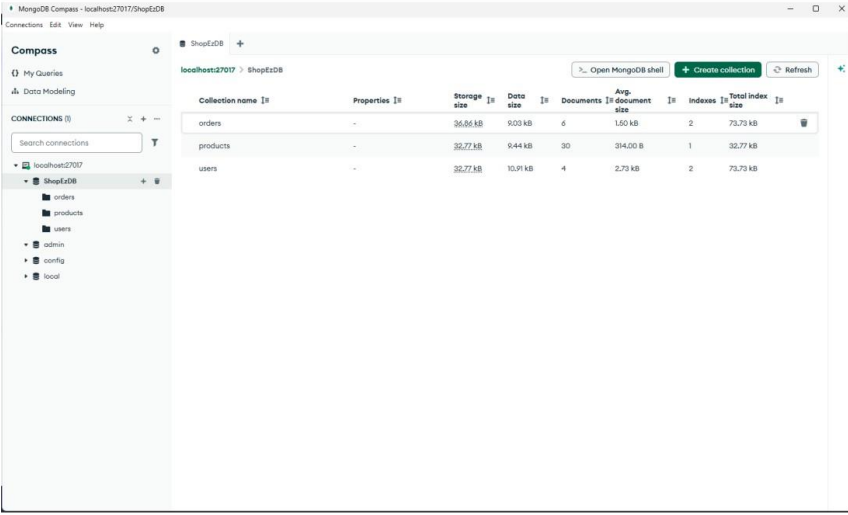
Order Status Tracking System

Order Status Tracking System



Backend Database – MongoDB Collection Overview

Backend Database - MongoDB Collection Overview



Customer Order Tracking & Transaction History



## Database View – User Credentials & Role Management

### Database View - User Credentials & Role Management

The image displays two screenshots. The top screenshot shows the MongoDB Compass interface for the 'ShopEzDB' database, specifically the 'users' collection. It lists four documents, each representing a user with fields for \_id, name, email, password, cartData, and date. The bottom screenshot shows the 'ShopEz Admin' interface, featuring a sidebar with 'Add Product' and 'Product List' options, and a main area titled 'All Products List' containing a table of products.

| PRODUCTS | TITLE                                      | OLD PRICE | NEW PRICE | CATEGORY | REMOVE |
|----------|--------------------------------------------|-----------|-----------|----------|--------|
|          | FLORAL PRINT V-NECK RUFFLE DETAIL WRAP TOP | \$137     | \$80      | WOMEN    | ✕      |
|          | SOLID COLOR A-LINE MAXI DRESS WITH POCKETS | \$144     | \$65      | WOMEN    | ✕      |
|          | CASUAL V-NECK BUTTON DOWN CARDIGAN SWEATER | \$139     | \$78      | WOMEN    | ✕      |
|          | HIGH WAIST SKINNY DENIM JEANS              | \$145     | \$84      | WOMEN    | ✕      |
|          | SLEEVELESS PLEATED CHIFFON MIDI DRESS      | \$131     | \$87      | WOMEN    | ✕      |
|          | WOMEN'S CLASSIC TRENCH COAT                | \$102     | \$52      | WOMEN    | ✕      |
|          | ATHLETIC YOGA LEGGINGS WITH POCKETS        | \$143     | \$71      | WOMEN    | ✕      |

Page 9

## Administrative Backend: Database Inventory & User Control

## 12. Identified Issues

- UI state reflects complex filter resets following manual page refresh.
- Performance scaling for large media uploads is being addressed via secondary cloud storage migration.

## 13. Future Roadmap

- Deployment of automated payment gateways (Stripe/Razorpay integration).
- Leveraging machine learning for personalized product suggestion engines.
- Expansion into Progressive Web App (PWA) territory for offline catalog access.